HTTP in Detail

Platform: TryHackMe
Category: Web Application Security
Room Link: https://tryhackme.com/room/httpindetail

1. Objective

Understand how HTTP works and how browsers communicate with web servers to access website content. Learn how HTTP and HTTPS differ, how requests and responses are structured, and how methods, status codes, and headers control communication. The room also covers cookies, their role in maintaining user sessions, and practical exercises involving different HTTP requests.

2. Key Concepts Learned

- **HTTP (HyperText Transfer Protocol):** A communication protocol used to transfer website content between clients and web servers.
- **HTTPS (HyperText Transfer Protocol Secure):** The secure version of HTTP that encrypts communication and helps verify the identity of the web server.
- **URL (Uniform Resource Locator):** An address that tells a browser how and where to access a particular resource.
- **URL Components:** Scheme, User, Host, Port, Path, Query String, and Fragment.
- **HTTP Request:** A message sent by a client to request information or perform an operation on a web server.
- **HTTP Response:** A message returned by the server containing the result of a client's request.
- **Request Line:** Contains the HTTP method, requested path, and protocol version.
- **Request Headers:** Additional information sent by the client to the server.
- **Response Headers:** Information returned by the server to explain how the response should be handled.
- **GET Method:** Used to retrieve information from a web server.
- **POST Method:** Used to submit information to a server and potentially create new records.
- **PUT Method:** Used to update information on a server.
- **DELETE Method:** Used to remove information or records.
- **HTTP Status Codes:** Three-digit codes that describe the result of an HTTP request.
- **Informational Responses (100–199):** Indicate that the request has been received and processing can continue.
- **Successful Responses (200–299):** Indicate that the request was successfully processed.
- **Redirection Responses (300–399):** Indicate that the requested resource has moved or requires redirection.
- **Client Errors (400–499):** Indicate problems with the client's request.
- **Server Errors (500–599):** Indicate problems while processing a request on the server.

- **HTTP Headers:** Provide additional information about requests and responses, including browser details, content types, and caching instructions.
- **Cookies:** Small pieces of data stored by the browser and sent back to the server with subsequent requests.
- **HTTP Statelessness:** HTTP does not automatically remember previous requests, so cookies are commonly used to maintain sessions and user preferences.
- **Session Cookies:** Values commonly used to identify authenticated users and maintain their login sessions.
- **Set-Cookie Header:** Sent by the server to instruct the browser to store a cookie.
- **Cookie Header:** Sent by the browser to provide previously stored cookies to the server.

3. Tools and Websites Covered

- **TryHackMe:** Provided the learning material and practical environment for understanding HTTP communication.
- **Web Browser:** Used to access websites and observe HTTP behaviour.
- **Browser Developer Tools:** Used to inspect network activity, requests, responses, and cookies.
- **Network Tab:** Used to examine resources requested by the browser and inspect their corresponding HTTP messages.
- **Cookies Tab:** Used to view cookie information associated with individual requests.
- **HTTP Request Emulator:** Used to manually select HTTP methods, configure URL parameters, submit body parameters, and observe server responses.
- **http.cat:** A reference website mentioned in the room for viewing different HTTP status codes.

4. Practical Work

- Studied the difference between HTTP and HTTPS and understood why encrypted communication is important when accessing websites.
- Explored the mock webpage in the first task and examined the issue presented in the exercise.
- Examined URL structures and understood how different components help browsers locate specific web resources.
- Analysed sample HTTP requests and responses to understand how clients communicate with web servers.
- Studied the request line and identified the purpose of the HTTP method, requested path, and protocol version.
- Examined HTTP request headers such as Host, User-Agent, Content-Length, Accept-Encoding, and Cookie.
- Reviewed response headers, including Set-Cookie, Cache-Control, Content-Type, and Content-Encoding.
- Compared GET, POST, PUT, and DELETE methods and understood how each method is used to interact with application resources.

- Studied different HTTP status code categories and examined common responses such as 200, 201, 301, 302, 400, 401, 403, 404, 405, 500, and 503.
- Explored how cookies are created through the Set-Cookie response header and returned to the server through subsequent requests.
- Opened browser Developer Tools and explored the Network tab to understand how website resources and HTTP requests can be inspected.
- Examined the Cookies section of network requests to understand how browsers send stored cookie information to web servers.
- Used the TryHackMe HTTP request emulator to practise different HTTP methods and configure request parameters.
- Sent a GET request to `/room` to retrieve the requested page.
- Sent a GET request to `/blog` with the `id` parameter set to `1`.
- Sent a DELETE request to `/user/1` to practise deleting a user resource.
- Sent a PUT request to `/user/2` with the username parameter set to `admin`.
- Sent a POST request to `/login` using the username `thm` and password `letmein`.

Practical HTTP Request Examples

1. GET Request

Used to retrieve the room page.

```
GET /room
```

2. GET Request with Query Parameter

Used to retrieve the blog article with ID 1.

```
GET /blog?id=1
```

3. DELETE Request

Used to remove the user with ID 1.

```
DELETE /user/1
```

4. PUT Request with Body Parameter

Used to update the username of the user with ID 2.

```
PUT /user/2

username=admin
```

5. POST Request with Login Parameters

Used to submit login credentials.

```
POST /login

username=thm&password=letmein
```

Note: These examples show the requested operations. The actual requests are submitted through the room's HTTP emulator, where URL and body parameters are configured using the settings button.

5. Important Security Observations

- **HTTPS protects transmitted information.** It encrypts communication between the browser and server, making intercepted traffic harder to read.
- **HTTP requests reveal useful information.** Examining request lines and headers helps understand how an application processes incoming traffic.
- **HTTP methods determine the intended operation.** GET, POST, PUT, and DELETE can interact with application resources in different ways.
- **Incorrect HTTP method handling can create security issues.** Applications should restrict methods according to the operations users are allowed to perform.
- **Status codes help identify application behaviour.** Responses such as 401, 403, 404, and 500 can provide useful information during security testing.
- **Response headers can disclose server information.** Headers such as Server may reveal details about the software running on the server.
- **Cookies can contain sensitive session information.** If an attacker obtains a valid session cookie, it may allow unauthorized access to an account.
- **HTTP is stateless.** Cookies help applications recognise returning users and maintain authenticated sessions across multiple requests.
- **Cookie security is important.** Session cookies should be protected using appropriate attributes such as Secure, HttpOnly, and SameSite.
- **Set-Cookie and Cookie serve different purposes.** Set-Cookie instructs the browser to store a cookie, while Cookie sends stored cookie data back to the server.
- **Developer Tools provide useful testing information.** The Network tab helps identify endpoints, request methods, parameters, headers, and server responses.
- **Request parameters should be validated.** Applications should not blindly trust values supplied through query strings or request bodies.

6. Security Recommendations

- Use HTTPS across the entire website to protect communication.
- Validate and sanitize all user-controlled input received through URLs, headers, and request bodies.
- Restrict HTTP methods to those required by the application.
- Implement proper authentication and authorization checks for sensitive operations.
- Protect session cookies using Secure, HttpOnly, and appropriate SameSite settings.
- Regenerate session identifiers after successful authentication to reduce session fixation risks.
- Avoid exposing sensitive server information through response headers.

- Configure appropriate error responses without revealing internal application details.
- Use suitable cache-control settings for sensitive pages and authenticated content.
- Monitor unusual HTTP requests and investigate unexpected access to application resources.

7. Key Learnings

This room helped me understand HTTP communication in more detail and how much information can be obtained by examining requests and responses. I learned how a browser communicates with a web server, how different HTTP methods are used, and how servers respond to different types of requests.

The practical exercises were particularly useful because they allowed me to work with GET, POST, PUT, and DELETE requests instead of only reading about them. I also became more familiar with configuring query and body parameters through an HTTP request emulator.

Another important part was understanding cookies and their role in maintaining user sessions. Learning how cookies are stored, sent, and inspected through browser Developer Tools gave me a better idea of how web applications identify users between requests.

My main takeaway is that **understanding HTTP traffic is an essential part of web application security testing**. Being able to inspect requests, modify parameters, understand server responses, and analyse cookies provides a strong foundation for identifying weaknesses during web application penetration testing.

Status: Complete